

CS101: Algorithms and Computational Complexity

Department of Computer Science & Engineering | Academic Handout

Page 1: Foundations of Asymptotic Analysis & Sorting

1.1 Asymptotic Notations and Formal Definitions

Asymptotic analysis evaluates algorithmic resource consumption as input size n approaches infinity. Three primary notations form the foundation of algorithm analysis:

- **Big-O Notation (O):** Represents an asymptotic upper bound. Formally, $f(n) = O(g(n))$ if there exist positive constants c and n_0 such that $0 \leq f(n) \leq c * g(n)$ for all $n \geq n_0$. It characterizes worst-case performance.
- **Big-Omega Notation (Ω):** Represents an asymptotic lower bound. Formally, $f(n) = \Omega(g(n))$ if there exist positive constants c and n_0 such that $0 \leq c * g(n) \leq f(n)$ for all $n \geq n_0$.
- **Big-Theta Notation (Θ):** Represents an asymptotically tight bound. Formally, $f(n) = \Theta(g(n))$ if and only if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.

Common complexity tiers ordered by growth rate: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$.

1.2 Comparison-Based Sorting Algorithms

DEFINITION: Quicksort is a divide-and-conquer sorting algorithm. It selects a pivot element, partitions the array into sub-arrays of elements less than and greater than the pivot, and recursively sorts the sub-arrays. On average, Quicksort runs in $O(n \log n)$ time and requires $O(\log n)$ auxiliary stack space. In the worst case (e.g. sorted input with deterministic first/last element pivot), Quicksort degrades to $O(n^2)$ time.

Mergesort consistently guarantees $O(n \log n)$ running time in best, average, and worst cases by dividing the array into halves, recursively sorting both, and merging the sorted sublists. However, standard array-based Mergesort requires $O(n)$ additional auxiliary memory for merging buffers. Mergesort is a stable sorting algorithm.

Heapsort constructs a binary heap data structure from the input array and iteratively extracts the maximum (or minimum) root element. Heapsort runs in guaranteed $O(n \log n)$ time in all cases and sorts in-place with $O(1)$ auxiliary space, though it is not a stable sort.

Page 2: Recurrence Relations, Master Theorem & Algorithmic Paradigms

1.3 Recurrence Relations and the Master Theorem

Divide-and-conquer algorithms often yield recurrence relations of the form: $T(n) = a * T(n/b) + f(n)$, where $a \geq 1$ represents the number of recursive subproblems, $b > 1$ represents the subproblem division factor, and $f(n)$ represents the cost of dividing and combining.

The Master Theorem provides asymptotic bounds across three principal cases by comparing $f(n)$ against $n^{\log_b(a)}$:

- Case 1: If $f(n) = O(n^{\log_b(a) - \epsilon})$ for some $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b(a)})$. The leaf work dominates.
- Case 2: If $f(n) = \Theta(n^{\log_b(a)} * (\log n)^k)$ for $k \geq 0$, then $T(n) = \Theta(n^{\log_b(a)} * (\log n)^{k+1})$. Work is evenly distributed across recursion levels.
- Case 3: If $f(n) = \Omega(n^{\log_b(a) + \epsilon})$ for $\epsilon > 0$ and regularity condition $a * f(n/b) \leq c * f(n)$ holds for $c < 1$, then $T(n) = \Theta(f(n))$. The root work dominates.

1.4 Dynamic Programming versus Greedy Method

Dynamic Programming (DP) solves optimization problems by decomposing them into overlapping subproblems and exhibiting optimal substructure. Solutions to subproblems are stored in a lookup table to eliminate redundant calculations. Top-down DP utilizes memoization, while bottom-up DP utilizes iterative tabulation.

The Greedy Method constructs a global solution by making a locally optimal choice at each step without reconsidering prior decisions. A greedy algorithm yields an optimal solution only if the problem exhibits both optimal substructure and the greedy-choice property (e.g. Fractional Knapsack, Dijkstra's Single-Source Shortest Path, Huffman Encoding). In contrast, the 0/1 Knapsack problem requires Dynamic Programming because the greedy choice does not guarantee global optimality.

Page 3: Hash Tables, Collision Resolution & Amortized Analysis

1.5 Hash Tables and Collision Handling Strategies

A Hash Table maps keys to bucket indices using a hash function $h(k)$. Under the Simple Uniform Hashing Assumption (SUHA), each key is equally likely to hash into any of the m available slots.

Collision resolution methods fall into two major categories:

1. Separate Chaining: Each bucket holds a linked list or self-balancing binary search tree of colliding entries. The load factor $\alpha = n / m$ can exceed 1.0. Search and deletion cost average $O(1 + \alpha)$.

2. Open Addressing: All elements reside directly within the hash table array. When a collision occurs, alternative slots are probed systematically until an empty bucket is encountered. Load factor α cannot exceed 1.0. Probing methods include:

- Linear Probing: $h(k, i) = (h'(k) + i) \bmod m$. Suffers from Primary Clustering where contiguous blocks of occupied slots grow and degrade search performance.

- Quadratic Probing: $h(k, i) = (h'(k) + c_1*i + c_2*i^2) \bmod m$. Avoids primary clustering but can suffer from Secondary Clustering.

- Double Hashing: $h(k, i) = (h_1(k) + i * h_2(k)) \bmod m$. Offers near-ideal pseudo-random probing, provided $h_2(k)$ is relatively prime to m .

Rehashing doubles table capacity and re-inserts all existing keys when load factor α exceeds a predefined threshold (typically 0.75), yielding amortized $O(1)$ insertion time.